

# **EDGE-SECURE LAB**

## **HYBRID NETWORK AND SECURITY**

**(Phase 2 — Hybrid Networking, SSH Trust Architecture &  
Secure Administrative Access)**

**BY**

**UDOFIA, EDINEN SUNDAY**  
**(Systems Administrator & I.T Support Engineer)**

**22<sup>ND</sup> APRIL, 2026**

|                                                    |    |
|----------------------------------------------------|----|
| 1. INTRODUCTION.....                               | 5  |
| 2. LAB ARCHITECTURE OVERVIEW.....                  | 5  |
| 3. HIGH-LEVEL NETWORK ARCHITECTURE.....            | 5  |
| 4. CORE NETWORKING CONCEPTS.....                   | 6  |
| 4.1 What is a Network Interface?.....              | 6  |
| 4.2 What is NAT?.....                              | 7  |
| 4.3 Why NAT Was Used.....                          | 7  |
| 4.4 What is Host-Only Networking?.....             | 8  |
| 4.5 Why Dual Interfaces Were Required.....         | 8  |
| 5. COMPLETE VIRTUALBOX NETWORK CONFIGURATION.....  | 9  |
| 5.1 edge01 Configuration.....                      | 9  |
| Adapter 1.....                                     | 9  |
| Adapter 2.....                                     | 9  |
| 5.2 backend01 Configuration.....                   | 10 |
| 6. STATIC IP ADDRESSING.....                       | 10 |
| 6.1 Why Static IPs Were Required.....              | 10 |
| 7. CONFIGURING edge01 STATIC IP (ROCKY LINUX)..... | 11 |
| 7.1 Verify Interfaces.....                         | 11 |
| 7.2 Configure Static IP.....                       | 11 |
| 7.3 Command Breakdown.....                         | 11 |
| 7.4 Activate Interface.....                        | 12 |
| 7.5 Verification.....                              | 12 |
| 8. CONFIGURING backend01 STATIC IP (UBUNTU).....   | 12 |
| 8.1 Edit Netplan Configuration.....                | 12 |
| 8.2 Full Configuration.....                        | 13 |
| 8.3 Configuration Breakdown.....                   | 13 |
| 8.4 Apply Configuration.....                       | 13 |
| 8.5 Verification.....                              | 14 |
| 9. UNDERSTANDING PACKET FLOW.....                  | 14 |
| 9.1 Internal Traffic Flow.....                     | 14 |
| 9.2 Internet Traffic Flow.....                     | 15 |
| 10. FIREWALL IMPLEMENTATION.....                   | 15 |
| 10.1 What is a Firewall?.....                      | 15 |
| 10.2 firewalld on Rocky Linux.....                 | 16 |
| Enable firewalld.....                              | 16 |
| Set Default Deny Policy.....                       | 16 |
| Allow SSH.....                                     | 16 |
| Allow HTTP.....                                    | 17 |
| Reload Rules.....                                  | 17 |
| 10.3 UFW on Ubuntu.....                            | 17 |

|                                                 |    |
|-------------------------------------------------|----|
| Enable Default Deny.....                        | 17 |
| Allow Outbound Traffic.....                     | 17 |
| Allow SSH.....                                  | 18 |
| Allow HTTP.....                                 | 18 |
| Enable Firewall.....                            | 18 |
| 11. SSH ARCHITECTURE.....                       | 18 |
| 11.1 What is SSH?.....                          | 18 |
| 11.2 Why SSH Keys Were Used.....                | 18 |
| 11.3 Ed25519 Key Generation.....                | 19 |
| Generated Files.....                            | 19 |
| 11.4 Cryptographic Authentication Flow.....     | 19 |
| 11.5 Copy Public Key to backend01.....          | 20 |
| 11.6 Azure SSH Integration.....                 | 20 |
| 12. WINDOWS SSH CONTROL PLANE ARCHITECTURE..... | 20 |
| 12.1 Engineering Objective.....                 | 20 |
| 12.2 Windows SSH Config File.....               | 21 |
| 12.3 Configuration Breakdown.....               | 21 |
| 12.4 Operational Result.....                    | 22 |
| 12.5 Security Understanding.....                | 22 |
| 13. TMUX PERSISTENT OPERATIONS.....             | 23 |
| 13.1 Why tmux Was Introduced.....               | 23 |
| 13.2 Install tmux.....                          | 23 |
| 13.3 Create Session.....                        | 23 |
| 13.4 Detach Session.....                        | 23 |
| 13.5 Reattach Session.....                      | 24 |
| 14. VERIFICATION PROCEDURES.....                | 24 |
| Verify Interfaces.....                          | 24 |
| Verify Routing.....                             | 24 |
| Verify Connectivity.....                        | 25 |
| Verify SSH.....                                 | 25 |
| Verify Azure Access.....                        | 25 |
| Verify firewalld.....                           | 25 |
| Verify UFW.....                                 | 25 |
| 15. TROUBLESHOOTING & FAILURE ANALYSIS.....     | 25 |
| 15.1 NAT Failure.....                           | 26 |
| 15.2 DNS Failure.....                           | 26 |
| 15.3 SSH Failure.....                           | 27 |
| 15.4 Internal Network Failure.....              | 27 |
| 15.5 Azure Firewall Failure.....                | 27 |
| 16. ENGINEERING OUTCOME.....                    | 28 |
| 17. FINAL ENGINEERING STATE.....                | 28 |



# 1. INTRODUCTION

This document captures the complete engineering implementation of the networking and secure access architecture for the EdgeSecure Hybrid Lab.

The objective of this phase was not merely to “connect machines,” but to understand:

- how packets move through systems,
- how virtualization networking works,
- how hybrid infrastructure is designed,
- how SSH trust relationships function,
- how firewalls enforce boundaries,
- how cloud security differs from local networking,
- and how engineers troubleshoot distributed systems.

This phase transformed isolated virtual machines into a functioning hybrid infrastructure environment.

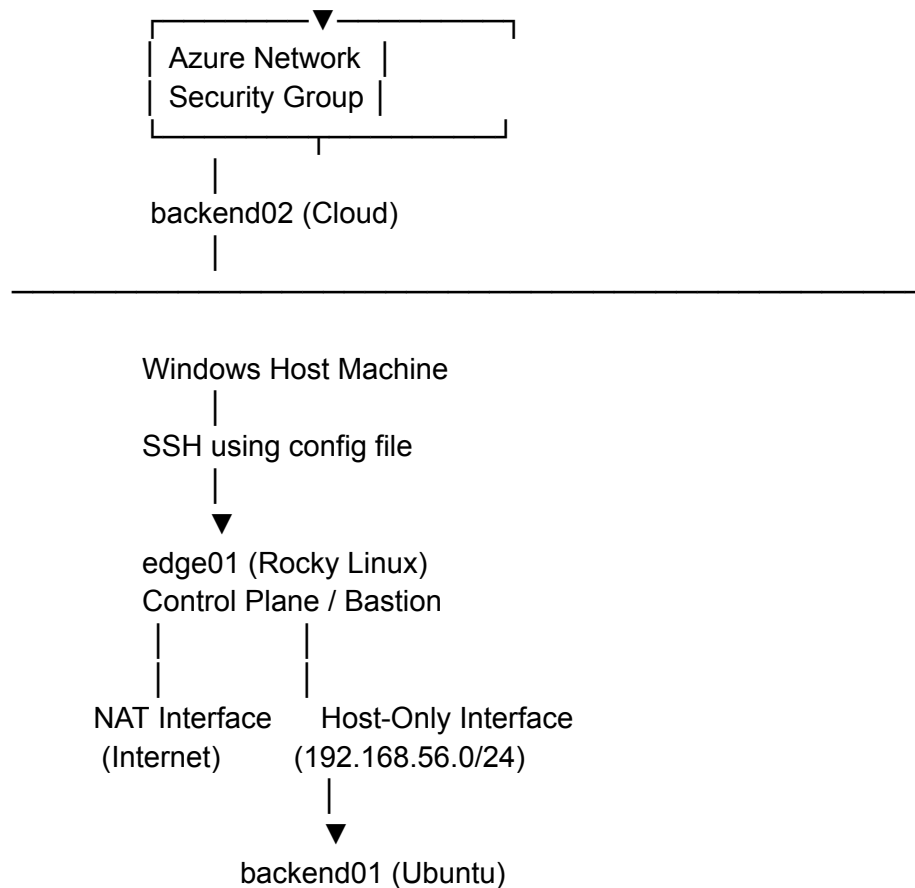
## 2. LAB ARCHITECTURE OVERVIEW

The environment consists of:

| Hostname  | Role                        | Platform        | OS            |
|-----------|-----------------------------|-----------------|---------------|
| edge01    | Control Node / Bastion Host | VirtualBox      | Rocky Linux   |
| backend01 | Local Backend Server        | VirtualBox      | Ubuntu Server |
| backend02 | Cloud Backend Server        | Microsoft Azure | Ubuntu Server |

## 3. HIGH-LEVEL NETWORK ARCHITECTURE

INTERNET  
|  
|



## 4. CORE NETWORKING CONCEPTS

### 4.1 What is a Network Interface?

A network interface is the operating system's representation of a network card (NIC).

Examples:

- physical Ethernet adapter
- Wi-Fi adapter
- virtual NIC created by VirtualBox

Linux interface examples:

| Interface | Purpose           |
|-----------|-------------------|
| enp0s3    | NAT adapter       |
| enp0s8    | Host-only adapter |

## 4.2 What is NAT?

NAT (Network Address Translation) allows a machine to access the internet without being directly exposed to the external network.

Architecture:

VM → VirtualBox NAT Engine → Host Machine → Internet

The VM receives internet access while remaining hidden behind the host system.

## 4.3 Why NAT Was Used

NAT was selected because:

- package installations require internet,
- updates require internet access,
- repositories must be reachable,
- VMs remain isolated from the physical LAN.

Without NAT:

- `dnf update`
- `apt update`
- package downloads

would fail.

## 4.4 What is Host-Only Networking?

Host-only networking creates a private isolated network shared only between:

- the host machine,
- VirtualBox VMs.

Typical subnet:

192.168.56.0/24

This network does NOT provide internet access.

Purpose:

- secure internal communication,
- backend traffic isolation,
- east-west traffic simulation,
- hybrid infrastructure routing.

## 4.5 Why Dual Interfaces Were Required

| Interface | Purpose                               |
|-----------|---------------------------------------|
| NAT       | Internet access                       |
| Host-only | Internal infrastructure communication |

Without dual interfaces:

- no separation of traffic,
- less realistic infrastructure design,
- harder troubleshooting,
- weaker security boundaries.

# 5. COMPLETE VIRTUALBOX NETWORK CONFIGURATION

---

## 5.1 edge01 Configuration

VirtualBox → Settings → Network

### Adapter 1

| Setting     | Value |
|-------------|-------|
| Enabled     | Yes   |
| Attached To | NAT   |

Purpose:

- internet access,
- package updates,
- Azure communication.

### Adapter 2

| Setting     | Value             |
|-------------|-------------------|
| Enabled     | Yes               |
| Attached To | Host-Only Adapter |

Purpose:

- private communication with backend01.

## 5.2 backend01 Configuration

Same dual-network architecture:

- NAT
- Host-only

This mirrors production multi-homed infrastructure systems.

## 6. STATIC IP ADDRESSING

### 6.1 Why Static IPs Were Required

Dynamic IPs (DHCP):

- can change after reboot,
- break automation,
- break SSH scripts,
- break Ansible inventories.

Static IPs ensure:

- stable routing,
- predictable infrastructure,
- consistent SSH access.

## 7. CONFIGURING edge01 STATIC IP (ROCKY LINUX)

### 7.1 Verify Interfaces

```
ip a
```

Expected interfaces:

```
enp0s3  
enp0s8
```

### 7.2 Configure Static IP

```
sudo nmcli con mod enp0s8 \  
ipv4.addresses 192.168.56.10/24 \  
ipv4.method manual
```

### 7.3 Command Breakdown

| Component        | Meaning            |
|------------------|--------------------|
| nmcli            | NetworkManager CLI |
| con              | connection         |
| mod              | modify             |
| enp0s8           | interface          |
| ipv4.addresses   | assign IP          |
| 192.168.56.10/24 | IP + subnet        |

ipv4.method manual    disable DHCP

## 7.4 Activate Interface

`sudo nmcli con up enp0s8`

This:

- brings interface UP,
- updates kernel routing tables,
- applies IP configuration.

## 7.5 Verification

`ip a`

Expected:

192.168.56.10/24

# 8. CONFIGURING backend01 STATIC IP (UBUNTU)

## 8.1 Edit Netplan Configuration

`sudo vim /etc/netplan/00-installer-config.yaml`

## 8.2 Full Configuration

```
network:  
  version: 2  
  ethernet:  
    enp0s8:  
      dhcp4: no  
      addresses:  
        - 192.168.56.11/24
```

## 8.3 Configuration Breakdown

| Parameter  | Meaning               |
|------------|-----------------------|
| network    | root config block     |
| version: 2 | netplan schema        |
| ethernet:  | interface definitions |
| enp0s8     | target interface      |
| dhcp4: no  | disable DHCP          |
| addresses  | static IP             |

## 8.4 Apply Configuration

```
sudo netplan apply
```

This:

- generates systemd-networkd configuration,
- pushes configuration into kernel networking stack.

## 8.5 Verification

ip a

Expected:

192.168.56.11

## 9. UNDERSTANDING PACKET FLOW

### 9.1 Internal Traffic Flow

Example:

ping 192.168.56.11

Actual path:

ping process

↓

Kernel networking stack

↓

Routing table lookup

↓

enp0s8 selected

↓

Packet sent to VirtualBox switch

↓

backend01 receives packet

↓

ICMP reply generated

↓

Response returned

## 9.2 Internet Traffic Flow

Example:

dnf update

Path:

edge01

↓

NAT interface

↓

VirtualBox NAT engine

↓

Host machine

↓

Home router

↓

Internet

↓

Rocky repositories

## 10. FIREWALL IMPLEMENTATION

### 10.1 What is a Firewall?

A firewall is a kernel-level packet filtering system.

Linux firewalls use:

- netfilter subsystem,
- nftables/iptables backend.

Purpose:

- allow,
- reject,

- drop traffic.

## 10.2 firewalld on Rocky Linux

### Enable firewalld

```
sudo systemctl enable --now firewalld
```

### Set Default Deny Policy

```
sudo firewall-cmd --set-default-zone=drop
```

This establishes:

- zero-trust default posture,
- all traffic denied unless explicitly allowed.

### Allow SSH

```
sudo firewall-cmd --add-service=ssh --permanent
```

Allows:

- TCP port 22.

## Allow HTTP

```
sudo firewall-cmd --add-service=http --permanent
```

Required later for:

- HAProxy,
- Apache.

## Reload Rules

```
sudo firewall-cmd --reload
```

Applies rules into kernel packet filter.

## 10.3 UFW on Ubuntu

### Enable Default Deny

```
sudo ufw default deny incoming
```

### Allow Outbound Traffic

```
sudo ufw default allow outgoing
```

## Allow SSH

```
sudo ufw allow ssh
```

## Allow HTTP

```
sudo ufw allow http
```

## Enable Firewall

```
sudo ufw enable
```

# 11. SSH ARCHITECTURE

## 11.1 What is SSH?

SSH (Secure Shell) provides:

- encrypted remote access,
- secure authentication,
- secure command execution.

## 11.2 Why SSH Keys Were Used

Password authentication:

- weaker,
- vulnerable to brute force,
- slower operationally.

SSH keys provide:

- stronger cryptography,
- passwordless authentication,
- automation capability.

## 11.3 Ed25519 Key Generation

Performed on edge01:

```
ssh-keygen -t ed25519
```

## Generated Files

| File                  | Purpose     |
|-----------------------|-------------|
| ~/.ssh/id_ed25519     | private key |
| ~/.ssh/id_ed25519.pub | public key  |

## 11.4 Cryptographic Authentication Flow

SSH login process:

Server sends challenge

↓

Client signs challenge using PRIVATE key

↓

Server verifies using PUBLIC key

↓

Authentication succeeds

No password transmitted.

## 11.5 Copy Public Key to backend01

`ssh-copy-id ubuntu@192.168.56.11`

This:

- logs into remote machine,
- creates `~/.ssh` if needed,
- appends public key into:  
`~/.ssh/authorized_keys`

## 11.6 Azure SSH Integration

The same public key from edge01 was uploaded into Azure during VM provisioning.

Azure automatically placed the key into:

`/home/azureuser/.ssh/authorized_keys`

Result:

- passwordless SSH from edge01 to backend02.

# 12. WINDOWS SSH CONTROL PLANE ARCHITECTURE

## 12.1 Engineering Objective

Instead of SSH-ing separately into every server, a centralized administrative workflow was implemented.

Architecture:

Windows Host

↓

edge01 (control plane)

↓

backend infrastructure

This mirrors:

- bastion hosts,
- jump servers,
- enterprise control nodes.

## 12.2 Windows SSH Config File

File location:

C:\Users\<username>\.ssh\config

Example configuration:

Host edge01

HostName 192.168.56.10

User sysadmin

IdentityFile C:\Users\<username>\.ssh\id\_ed25519

## 12.3 Configuration Breakdown

| Directive | Meaning |
|-----------|---------|
|-----------|---------|

|      |       |
|------|-------|
| Host | alias |
|------|-------|

HostName    target IP  
User        SSH username  
IdentityFile   private key location

## 12.4 Operational Result

Typing:

```
ssh edge01
```

automatically expands to:

```
ssh -i key sysadmin@192.168.56.10
```

## 12.5 Security Understanding

The private key was copied from edge01 into Windows.

This means:

- Windows now possesses edge01 identity,
- Windows can authenticate as edge01.

Lab-safe:

- acceptable in controlled environment.

Production concern:

- private keys must be heavily protected,
- often encrypted with passphrases,
- sometimes hardware-backed.

# 13. TMUX PERSISTENT OPERATIONS

## 13.1 Why tmux Was Introduced

Without tmux:

SSH disconnect

↓

session terminates

↓

processes die

With tmux:

SSH disconnect

↓

tmux survives

↓

operations continue

## 13.2 Install tmux

```
sudo dnf install -y tmux
```

## 13.3 Create Session

```
tmux new -s controlplane
```

## 13.4 Detach Session

Keyboard:

CTRL+b  
then d

## 13.5 Reattach Session

`tmux attach -t controlplane`

# 14. VERIFICATION PROCEDURES

## Verify Interfaces

`ip a`

Expected:

- NAT interface,
- host-only interface.

## Verify Routing

`ip route`

Expected:

- default route,
- local subnet route.

## **Verify Connectivity**

`ping 192.168.56.11`

## **Verify SSH**

`ssh ubuntu@192.168.56.11`

## **Verify Azure Access**

`ssh azureuser@<public-ip>`

## **Verify firewalld**

`sudo firewall-cmd --list-all`

## **Verify UFW**

`sudo ufw status`

# **15. TROUBLESHOOTING & FAILURE ANALYSIS**

## 15.1 NAT Failure

Symptom:

- no internet,
- dnf failures,
- repository errors.

Cause:

- NAT adapter disabled.

Fix:

- enable NAT adapter in VirtualBox.

## 15.2 DNS Failure

Symptom:

ping 8.8.8.8 works  
ping google.com fails

Cause:

- DNS resolution issue.

Fix:

```
sudo vim /etc/resolv.conf
```

Add:

```
nameserver 8.8.8.8  
nameserver 1.1.1.1
```

## 15.3 SSH Failure

Symptom:

Permission denied (publickey)

Possible causes:

- missing authorized\_keys,
- wrong permissions,
- wrong key path,
- incorrect username.

Fix:

- re-run ssh-copy-id,
- verify ~/.ssh permissions.

## 15.4 Internal Network Failure

Simulated failure:

```
sudo nmcli con down enp0s8
```

Effect:

- interface DOWN,
- route removed,
- packets cannot leave system.

Fix:

```
sudo nmcli con up enp0s8
```

## 15.5 Azure Firewall Failure

Symptom:

- SSH timeout.

Cause:

- port 22 blocked in Azure NSG.

Fix:

- restore inbound port rule.

## 16. ENGINEERING OUTCOME

At the completion of this phase, the infrastructure achieved:

| Capability                        | Status      |
|-----------------------------------|-------------|
| Hybrid networking                 | Implemented |
| Static IP addressing              | Implemented |
| Dual-interface architecture       | Implemented |
| Secure SSH trust model            | Implemented |
| Cloud connectivity                | Implemented |
| Host-based firewalling            | Implemented |
| Centralized administrative access | Implemented |
| Persistent operational sessions   | Implemented |

## 17. FINAL ENGINEERING STATE

The environment now operates as:

- a hybrid infrastructure lab,
- a secure administrative control plane,
- a distributed networked system,
- a cloud-integrated Linux environment,
- a production-style operational architecture.

This phase established the networking and trust foundations required for:

- Ansible automation,
- HAProxy load balancing,
- centralized logging,
- disaster recovery,
- security monitoring,
- and infrastructure resilience.